Color Segmentation and Shape Classification

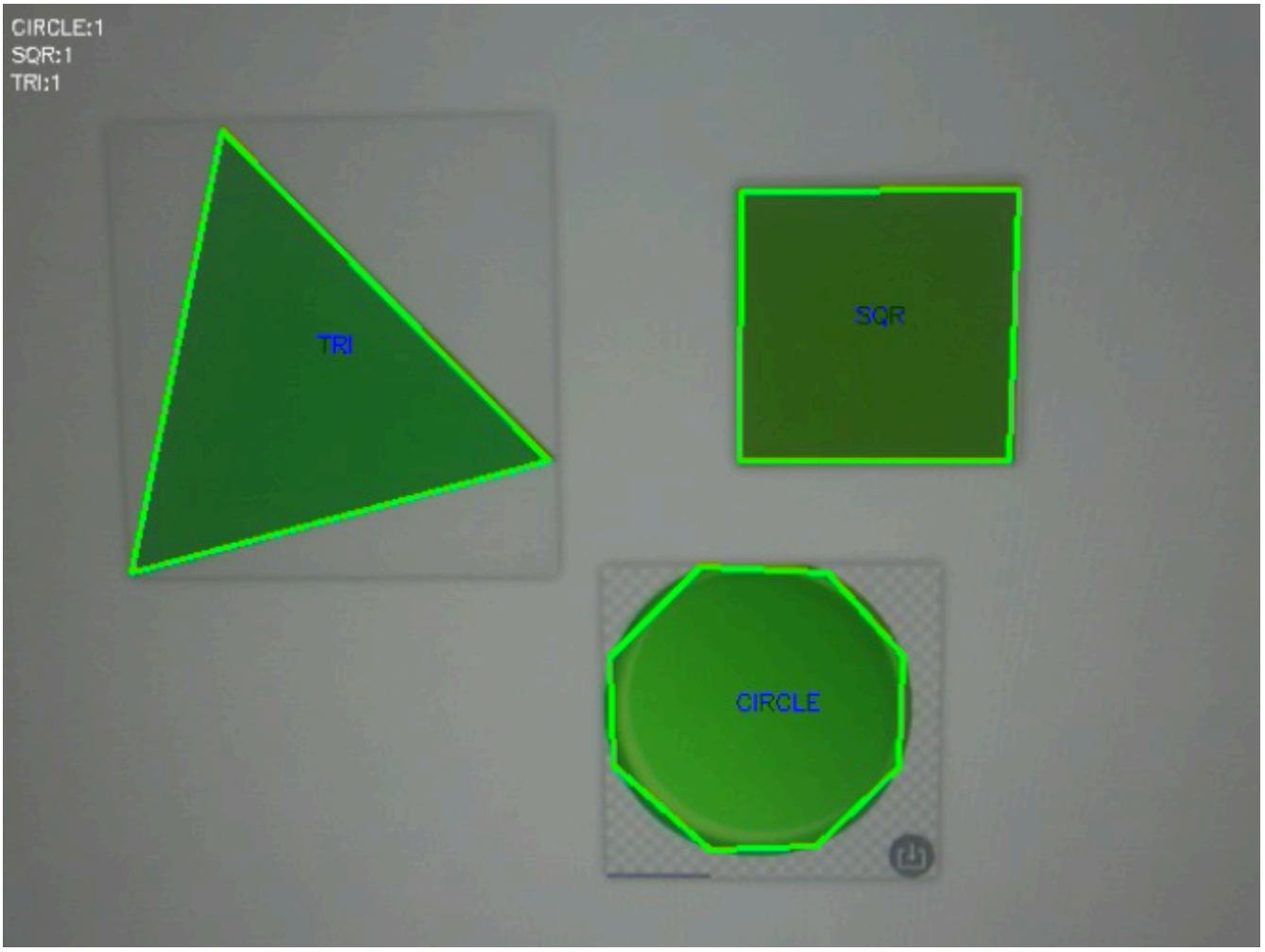
Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. Download from the official Canaan website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

Example Results

Run the example code for this section: 03.camera_shape_classify.py.

This section extracts green regions from real-time video, cleans mask noise, finds external contours, and classifies triangles, squares, rectangles, circles, and general polygons based on the number of vertices after contour approximation.



Principles

Why Use HSV for Color Segmentation

BGR mixes color and brightness information together, making thresholds difficult to set when illumination changes. HSV separates color into Hue H, Saturation S, and Value V, making it more suitable for building masks based on color ranges.

The example uses the following green range:

```
CL = np.array([35, 60, 60])
CH = np.array([85, 255, 255])
```

H limits the green hue, and the lower limits of S and V are used to exclude grayish-white areas and overly dark areas.

Shape Classification Process

```
BGR image
  ↓ cvtColor
HSV image
  ↓ inRange
Green binary mask
  ↓ Opening
Denoised mask
  ↓ findContours
External contours
  ↓ Area filtering, approxPolyDP
Polygon vertices
  ↓ Vertex count and aspect ratio
Shape category
```

Classification rules are as follows:

Condition	Category	Description
3 vertices	§TRI§	Triangle
4 vertices, aspect ratio 0.85~1.15	§SQR§	Approximate square
4 vertices, other aspect ratios	§RECT§	Rectangle
5~7 vertices	§POLY§	General polygon
8 or more vertices	§CIRCLE§	Approximate circle

"Circle" is only approximated by vertex count, not strict circularity detection. Complex contours may also be classified into this category.

Code Overview

Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Set Thresholds and Minimum Area

```
CL = np.array([35, 60, 60])
CH = np.array([85, 255, 255])
MIN_A = 300
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
```

`MIN_A` is used to exclude noise contours with area less than 300 pixels. The 3×3 elliptical kernel is pre-allocated outside the loop.

Define Shape Classifier

```
def classify_shape(approx):
    count = len(approx)
    if count == 3:
        return "TRI"
    if count == 4:
        x, y, w, h = cv2.boundingRect(approx)
        ratio = float(w) / (h or 1)
        return "SQR" if 0.85 < ratio < 1.15 else "RECT"
    if count >= 8:
        return "CIRCLE"
    if count >= 5:
        return "POLY"
    return "?"
```

`h or 1` prevents division by zero caused by abnormal contours. `boundingRect()` returns an axis-aligned rectangle. After the target is rotated, the aspect ratio changes, so a tilted square may be classified as a rectangle.

Color Segmentation and Denoising

```
hsv = cv2.cvtColor(img_np, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, CL, CH)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
```

`inRange()` sets pixels within the range to 255 and others to 0. Opening operation removes isolated white spots in the mask.

Find and Filter Contours

```
contours, _ = cv2.findContours(
    mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
    area = cv2.contourArea(contour)
    if area < MIN_A:
        continue
```

`RETR_EXTERNAL` only returns the outermost contours, and `CHAIN_APPROX_SIMPLE` compresses redundant points on straight line segments. The source code processes at most 50 valid contours to prevent complex scenes from occupying too much time and memory.

Polygon Approximation and Result Drawing

```
perimeter = cv2.arcLength(contour, True)
approx = cv2.approxPolyDP(contour, 0.03 * perimeter, True)
shape = classify_shape(approx)
x, y, w, h = cv2.boundingRect(contour)

cv2.drawContours(img_np, [approx], -1, (0, 255, 0), 2)
cv2.putText(
    img_np, shape,
    (x + w // 2 - 12, y + h // 2),
    cv2.FONT_HERSHEY_SIMPLEX, 0.4,
    (0, 0, 255), 1
)
```

The approximation error is set to 3% of the perimeter. Larger error means fewer vertices; smaller error means contour noise is more easily retained as extra vertices.

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **HSV range**: First sample the HSV value of the target under actual illumination, then adjust the upper and lower limits. Do not guess thresholds based solely on color names.
- **Red segmentation**: Red spans the beginning and end of the H axis. It usually requires two mask segments with low hue and high hue, then use `bitwise_or()` to combine them.
- **MIN_A**: Increase when there are many false detections of small spots; decrease when small targets are

filtered out.

- **Approximation ratio 0.03**: Can be appropriately reduced when circles are identified as polygons; can be appropriately increased when there are too many noise vertices.
- **Square aspect ratio**: The current range is 0.85~1.15. When targets have perspective or rotation, you should further judge based on side length, angle, or minimum enclosing rectangle.
- **Circle reliability**: In actual projects, you can add a circularity condition of `4π×area÷perimeter²` to avoid judgment based solely on vertex count.